

End-to-End CI/CD Pipeline

AWS EC2 · Terraform · Jenkins · Docker · GitHub · Nginx

Project Report

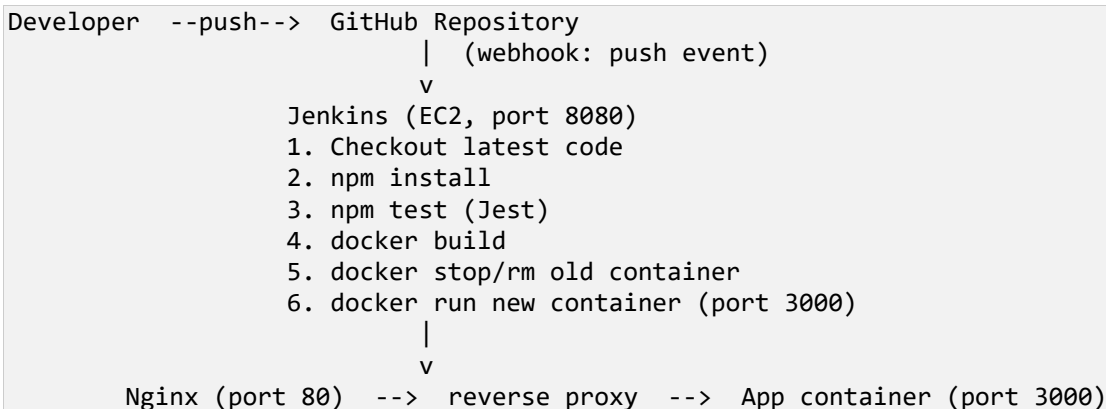
1. Project Overview

This project implements a fully automated CI/CD pipeline for a Node.js/Express web application. The goal was to demonstrate practical, hands-on proficiency across the core DevOps toolchain — Infrastructure as Code, continuous integration, containerization, and automated deployment — by building a real, working system end to end rather than following a purely theoretical exercise.

On every push to the application's GitHub repository, the pipeline automatically pulls the latest code, installs dependencies, runs the automated test suite, builds a new Docker image, and redeploys the running container — with zero manual intervention required after the initial setup.

2. Architecture

The system is built around a single AWS EC2 instance acting as both the CI/CD server and the application host, fronted by Nginx as a reverse proxy.



3. Technology Stack

Component	Purpose
AWS EC2	Compute host for Jenkins, Docker, and the running application
Terraform	Infrastructure as Code — provisions the EC2 instance, security group, key pair, default VPC, and Elastic IP
Jenkins	CI/CD orchestration — pipeline defined declaratively in a Jenkinsfile
Docker	Packages the application and its runtime into a portable container image
GitHub	Source control and webhook trigger for automated builds
Nginx	Reverse proxy exposing the app on port 80
Node.js / Express	The application itself
Jest	Automated test framework, run as part of the pipeline

4. Infrastructure Provisioning (Terraform)

All AWS infrastructure is defined declaratively and version-controlled, rather than being created manually through the AWS Console. The Terraform configuration provisions:

- A security group allowing inbound traffic on ports 22 (SSH), 80 (HTTP), 8080 (Jenkins UI), and 3000 (app, for direct testing)

- An EC2 key pair, generated locally and referenced by Terraform for SSH access
- A default VPC (created explicitly via `aws_default_vpc`, since the AWS account had no default VPC in the target region)
- A t2.micro EC2 instance running Amazon Linux 2, bootstrapped via a user-data script
- An Elastic IP, attached to the instance so its public address remains fixed across stops and restarts

The bootstrap (user-data) script installs Docker, Git, Nginx, and Java, and configures Jenkins to start on boot.

5. CI/CD Pipeline (Jenkins)

The pipeline is defined as code in a Jenkinsfile committed to the application repository, and is picked up by Jenkins via "Pipeline script from SCM." It consists of four stages:

1. Checkout — pulls the latest commit from the main branch using stored GitHub credentials
2. Install & Test — runs `npm install` followed by `npm test` (Jest)
3. Build Docker Image — builds and tags a new image from the application's Dockerfile
4. Deploy — stops and removes the previous container, then starts a new one from the freshly built image

A GitHub webhook (configured for the push event) notifies Jenkins immediately after every push, triggering the pipeline automatically — no manual "Build Now" is required in normal operation.

6. Containerization (Docker)

The application is packaged using a minimal Node.js Alpine-based Dockerfile, exposing port 3000. Each successful pipeline run produces a new tagged image and replaces the running container, giving a clean, reproducible deployment on every change.

7. Reverse Proxy (Nginx)

Nginx is configured as a reverse proxy listening on port 80, forwarding requests to the application container on port 3000. This avoids exposing the application's internal port directly to the internet and mirrors how a production deployment would typically be fronted.

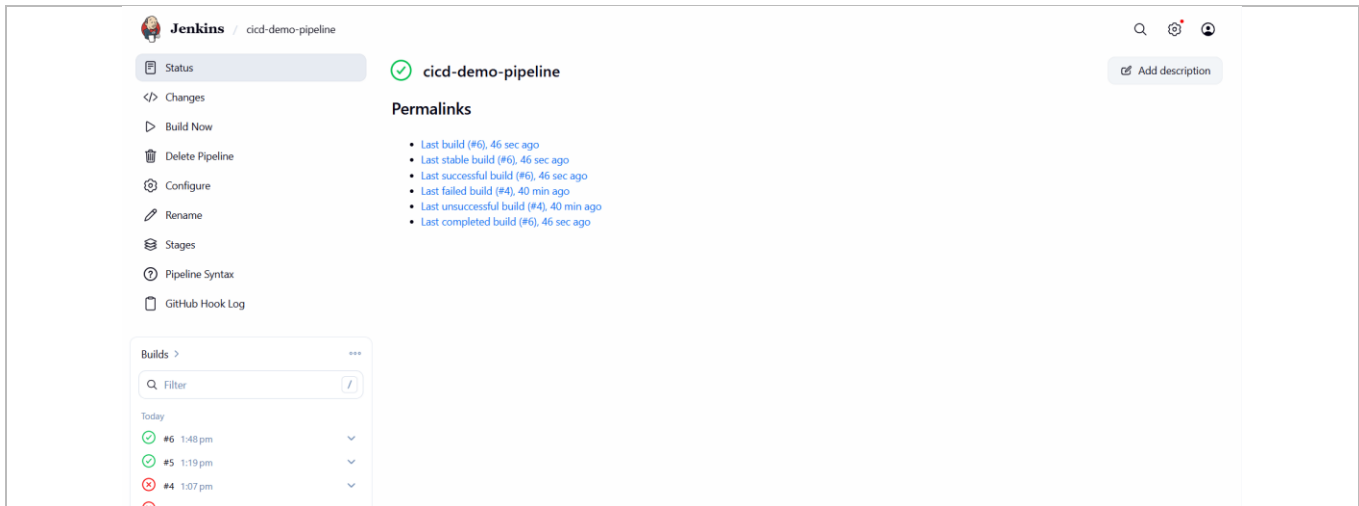
8. Challenges Encountered and Resolved

Several real-world obstacles came up during setup, each of which was diagnosed and resolved directly rather than worked around:

Issue	Root cause	Resolution
Security group creation failed	AWS account had no default VPC in the region	Added an explicit <code>aws_default_vpc</code> resource in Terraform
Jenkins failed to start	Installed Jenkins version required Java 21; server had Java 17	Installed Amazon Corretto 21 and switched the system default with alternatives
npm: command not found in pipeline	Node.js was never installed on the EC2 server itself	Installed Node via <code>nvm</code> ; AL2's outdated <code>glibc</code> required stepping down to Node 16
Jenkins still couldn't see npm	Symlinks pointed into a home directory with 700 permissions, unreadable by the jenkins user	Copied the Node binaries to <code>/usr/local</code> and symlinked from <code>/usr/bin</code> instead
Jest tests failed under Node 16	Installed Jest 30 requires Node 18+	Pinned Jest and Supertest to Node-16-compatible versions in <code>package.json</code>
App unreachable after instance reboot	EC2 public IP changes on every stop/restart; Docker container did not restart automatically	Attached a Terraform-managed Elastic IP and set the container's restart policy to <code>unless-stopped</code>

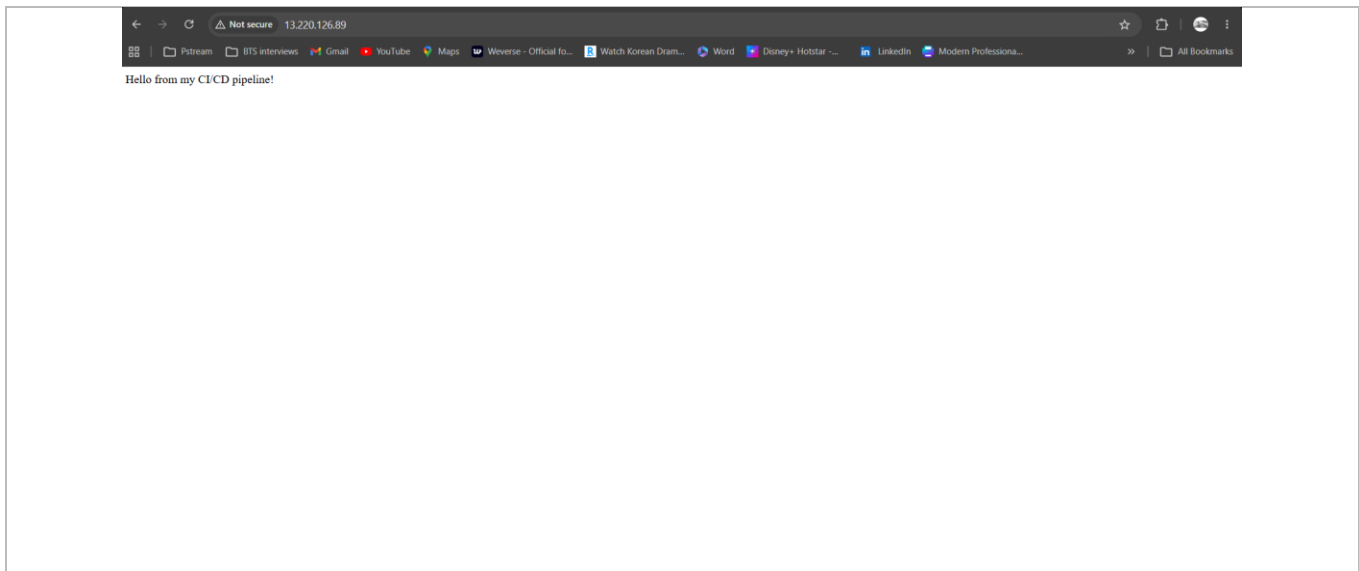
9. Results

Jenkins pipeline — automatic build triggered by a GitHub push



Build #6 completed successfully, triggered automatically by the GitHub webhook (no manual "Build Now").

Application served through Nginx on port 80



The deployed application, reachable on the standard HTTP port via the Nginx reverse proxy.

10. Outcome

The finished system demonstrates a complete, working automation loop: a code change pushed to GitHub is automatically tested, containerized, and deployed to a live server within roughly a minute, with the running application served through a reverse proxy on the standard HTTP port. All infrastructure is reproducible from version-controlled Terraform configuration, and the pipeline itself is defined as code alongside the application.

Skills demonstrated: Infrastructure as Code, CI/CD pipeline design, Docker containerization, GitHub webhook integration, Linux systems administration and troubleshooting, and reverse proxy configuration.